

Работа со сложными структурами данных в словарях

Работа со сложными структурами данных — это то, что превращает простого программиста в разработчика систем. Вложенность позволяет нам описывать мир таким, какой он есть: многослойным и структурированным.



1. Списки внутри словаря (Значения-коллекции)

Это самый частый случай. Мы используем его, когда одному ключу соответствует несколько однотипных элементов.

Пример: Система управления успеваемостью студентов в университете (например, в КазНУ или КБТУ).

```
# Ключ - имя студента, значение - список его оценок
university_group = {
    "Данияр": [90, 85, 92],
    "Айлин": [88, 92, 95],
    "Серик": [70, 75, 72]
}

# Как работать:
university_group["Данияр"].append(100) # Добавляем новую
оценку в список
print(f"Оценки Данияра: {university_group['Данияр']}")
```



2. Словари внутри словаря (Глубокая вложенность)

Используется для создания иерархий. Это стандарт де-факто для передачи данных через интернет (формат JSON).

Пример: База данных филиалов компании в разных городах Казахстана.

```
branches = {
    "Astana": {
        "address": "пр. Мангилик Ел, 55",
        "staff_count": 25,
        "manager": "Аскар"
    },
    "Almaty": {
        "address": "ул. Абая, 10",
```

```

        "staff_count": 40,
        "manager": "Зарина"
    }
}

# Как получить доступ:
manager_name = branches["Almaty"]["manager"]
print(f"Менеджер в Алматы: {manager_name}")

```

🔑 3. Кортежи как ключи (Неизменяемость)

Это важный момент. **Списки нельзя использовать в качестве ключей**, потому что они могут меняться (изменяемые). А **кортежи** — можно, потому что они стабильны.

Пример: Географические координаты городов (широта и долгота).

```

# Ключ - кортеж (широта, долгота), значение - название города
geo_locations = {
    (51.16, 71.42): "Астана",
    (43.22, 76.85): "Алматы",
    (42.32, 69.59): "Шымкент"
}

# Поиск города по координатам
coords = (51.16, 71.42)
print(f"По координатам {coords} находится город {geo_locations[coords]}")

```

Вложенные структуры данных — это способ организовать хаос.

- **Списки в значениях** позволяют группировать данные.
- **Словари в значениях** создают структуру и иерархию.
- **Кортежи в ключах** позволяют использовать сложные объекты (например, точки в пространстве) как уникальные идентификаторы.

Главное правило: **ключ должен быть неизменяемым**, а значение может быть чем угодно.

Если словарь описывает **один** объект (например, одного человека), то список словарей описывает **множество** таких объектов (весь коллектив).



1. Словари внутри списков (База данных)

Это стандартный способ хранения табличных данных. Каждая строка таблицы — это словарь, а вся таблица — это список.

Пример: Каталог товаров в интернет-магазине.

```
# Список, содержащий словари
products = [
    {"name": "iPhone 15", "price": 550000, "in_stock":
    True},
    {"name": "Samsung S23", "price": 420000, "in_stock":
    False},
    {"name": "Xiaomi 13", "price": 300000, "in_stock":
    True}
]

# Как работать:
# Доступ к цене второго товара (Samsung):
print(products[1]["price"])

# Поиск всех товаров, которые есть в наличии:
available_products = [p["name"] for p in products if
p["in_stock"]]
```



2. Словари внутри кортежей (Защищенные данные)

Кортежи используются, когда мы хотим создать **неизменяемую последовательность** объектов. Если мы положим словари в кортеж, мы не сможем добавить новые словари или удалить существующие, хотя содержимое самих словарей внутри кортежа всё еще можно будет менять.

Пример: Настройки системы, которые нельзя удалять во время работы.

```
# Кортеж со словарями настроек
SYSTEM_CONFIG = (
    {"module": "auth", "status": "active"},
    {"module": "database", "status": "connected"}
)
```

```
# Мы можем изменить статус:
SYSTEM_CONFIG[0]["status"] = "maintenance"

# Но мы НЕ можем добавить новый модуль:
# SYSTEM_CONFIG += ({ "module": "logs" },) -- это создаст
новый кортеж, а не изменит старый.
```

Практическое применение: JSON

Почти всё, что вы видите в интернете (лента Instagram, сообщения в WhatsApp, курсы валют), передается в виде комбинации **Списков и Словарей**.

Пример структуры ответа от погодного сервера:

```
weather_report = {
    "city": "Astana",
    "forecast": [ # Список словарей с прогнозом по
дням
        {"day": "Monday", "temp": -15, "wind": "5m/s"},
        {"day": "Tuesday", "temp": -12, "wind": "3m/s"},
        {"day": "Wednesday", "temp": -18, "wind":
"10m/s"}
    ]
}
```

Примеры использования на производстве

1. **Е-commerce (Kaspi.kz / Magnum):** Категория -> Подкатегория -> Список товаров.
2. **Игровая индустрия:** Координаты игрока (x, y, z) как ключ в словаре объектов мира.
3. **Банковское дело:** Словарь, где ключ — ИИН, а значение — словарь с историей транзакций (списки).

Задачи для закрепления (Лёгкий уровень)

Задача 1: Моё хобби

Создайте словарь person, где ключами будут name и hobbies. Значением для hobbies должен быть **список** из трех ваших любимых занятий. Добавьте в этот список четвертое хобби программно.

Задача 2: Телефонная книга семьи

Создайте словарь `family`, где ключи — это степени родства (например, "Мама", "Папа"), а значения — **словари** с именем и номером телефона. Выведите на экран имя мамы.

Задача 3: Шахматная доска

Создайте словарь `board`, где ключами будут **кортежи** с координатами клеток (например, (1, 1), (1, 2)), а значениями — названия фигур на них. Поставьте на клетку (1, 1) "Белую Ладью".

Задача 4: Список покупок

Дан словарь `fridge = {"молоко": 2, "яйца": 10}`. Измените его структуру так, чтобы значением каждого продукта был **список**, где первый элемент — количество, а второй — срок годности (строка).

Задача 5: Обновление данных

Дан словарь:

```
kazakhstan = {
    "regions": {
        "Almaty": ["Esik", "Kaskelen"],
        "Akmola": ["Stepnogorsk"]
    }
}
```

Добавьте в список городов Акмолинской области город "Kokshetau".

Задача 6: Список студентов

Создайте список `students`, состоящий из трех словарей. У каждого студента должны быть ключи: `name` (имя) и `points` (целое число). Напишите цикл, который выводит только имена тех студентов, у которых больше 80 баллов.

Задача 7: Очередь заказов

Создайте кортеж `orders`, содержащий два словаря. В каждом словаре есть `order_id` и `status`. Измените статус первого заказа на "Delivered".



Общий вывод

Связка **Списки + Словари + Кортежи** позволяет строить структуры любой сложности:

1. **Списки** (`[]`) — используйте для упорядоченных коллекций (очереди, списки товаров).
2. **Словари** (`{}`) — используйте для описания свойств конкретного объекта (имя, цена, характеристики).
3. **Кортежи** (`()`) — используйте для неизменяемых наборов или в качестве ключей в словарях.

Комбинируя их, вы можете описать абсолютно любую информацию: от состава атома до структуры целой галактики.